OrgLink — Flutter App Wireframe, Firebase System Architecture, and Functional Design

1. Project Title

OrgLink: Student Organization Management App

2. Project Overview

Using the provided BarkadaBase research paper as the reference for visualization style, system flowcharts, and organizational management concepts, design an updated mobile application called OrgLink.

OrgLink is an Android application designed to solve disorganized student organization management by providing a centralized platform for membership management, event creation, event registration, attendance verification, organizational calendars, digital badges, and event ratings.

The application must encourage student participation through a badge-based gamification system while allowing organization managers and authorized officers to manage their organizations efficiently.

Unlike the original BarkadaBase web-based system, OrgLink will use Flutter and Dart for Android application development, Firebase Authentication for Google sign-in, Cloud Firestore for data management, and Git/GitHub for version control and team collaboration.

The output must be a realistic initial design suitable for a Computer Engineering capstone project. It must include actual mobile wireframes, navigation flows, database structures, asset inventories, role-based permissions, page functions, and Firebase integration diagrams.

Do not design only a homepage. Create the complete application, covering onboarding, authentication, organization discovery, membership, event management, attendance, badges, ratings, notifications, and account settings.

3. Technology Stack

Use the following technologies:

- Flutter and Dart — Android application development.
- Visual Studio Code — primary code editor.
- Android Studio — Android SDK management, emulator, device testing, and native Android troubleshooting.
- Firebase Authentication — Google sign-in, account identity, sign-out, and authentication management.
- Cloud Firestore — users, organizations, memberships, events, registrations, attendance, badges, ratings, notifications, and related records.
- Firebase Cloud Storage — optional organization logos and event posters, subject to project billing and storage configuration.
- Firestore Security Rules — database authorization and validation.

- Firestore composite indexes — queries that require additional indexing.
- Firebase Emulator Suite — local testing of supported Firebase services.
- Provider and ChangeNotifier — application state management.
- go_router — application navigation and route protection.
- flutter_lints — code quality and consistent Dart conventions.
- Git and GitHub — version control and collaboration.

Separate development and production Firebase environments:

- `orglink-dev`: development data, test users, sample organizations, and test events.
- `orglink-prod`: final demonstration and production data.

Never use real student information in development fixtures. Keep development and production configuration separate.

4. User Roles and Authorization Model

The system must distinguish between a platform-level role and an organization-specific membership role.

A. Platform Roles

Platform Administrator (`platformAdmin`)

Responsible for platform-wide administration, authorized organization oversight, and leadership-transfer requests.

Capabilities:

- View platform-level statistics.
- Manage organization records when authorized.
- Provision and verify organization managers.
- Review manager-transfer requests.
- Monitor system announcements and operational issues.
- Review appropriate platform-level activity logs.

The platform administrator must be securely provisioned. Public users must not be able to assign this role to themselves.

Regular User (`user`)

The default role assigned to an authenticated account.

Capabilities:

- Maintain permitted profile information.
- Browse public organizations.
- Request membership.
- Register for eligible events.
- View personal attendance history and earned badges.
- Submit one rating per attended event.

- Receive relevant notifications.

A regular user can have different membership roles in different organizations.

B. Organization Membership Roles

Organization Manager (`manager`)

- Manage the assigned organization's information.
- Approve, reject, or remove memberships.
- Appoint officers and assign permitted responsibilities.
- Create, edit, publish, cancel, and archive events.
- View attendance reports.
- Configure or manage badge-award rules.
- Review ratings and engagement summaries.
- Request a manager transfer when leadership changes.

Managers must not automatically have platform-wide administrator privileges.

Organization Officers (`officer`)

Officers must have assigned responsibilities. Include the following officer types:

- **President:** oversees organizational activities and leadership coordination.
- **Vice President:** assists the president and coordinates committees or projects.
- **Secretary:** manages the membership roster, membership records, and meeting documentation.
- **Treasurer:** responsible for financial-related records only if financial functionality is implemented.
- **Event Coordinator:** creates and manages events according to assigned permissions.
- **Attendance Officer:** verifies attendance and reviews attendance summaries.
- **Public Information Officer (PIO):** manages announcements and promotional content when authorized.
- **Committee Head:** coordinates a designated committee and its assigned members.

Do not grant every officer unrestricted access to all functions. The manager must assign specific permissions to officers, and the application must enforce those permissions.

Regular Member (`member`)

- View and update permitted profile fields.
- Browse organization information.
- Request membership and monitor approval status.
- Register for events.
- View their own registration and attendance history.
- Collect badges after qualifying attendance.
- View their personal badge collection.
- Submit one rating and optional comment after attending an event.

Membership Statuses

Use these organization-specific membership statuses:

- `pending`
- `active`
- `rejected`
- `removed`

Membership roles and statuses must be stored separately from the platform-level user role.

An authenticated account may be a member of several organizations, with a different membership role and status in each organization.

5. Required Application Screens and Wireframes

Create a low-fidelity mobile wireframe for each screen. Use Android phone proportions and a consistent navigation system.

A. Onboarding and Authentication

Screen 1: Splash Screen

- OrgLink logo.
- Application name.
- Loading indicator.
- Authentication state check.

Desired output: Initialize the application and determine whether the user should see onboarding, sign-in, or the appropriate dashboard.

Screen 2: Onboarding

- Brief explanation of organization discovery.
- Event participation and attendance tracking.
- Digital badges and member engagement.
- Continue and sign-in buttons.

Desired output: Introduce the main value of OrgLink.

Screen 3: Google Sign-In

- OrgLink branding.
- Continue with Google button.
- Terms and privacy links.
- Authentication loading and error states.

Desired output: Authenticate users through Firebase Authentication and retrieve the authenticated user's UID.

Screen 4: Complete Student Profile

- Full name.

- Student number.
- Year level and section.
- Optional profile photo.
- Email address from the authenticated account where available.
- Save profile button.

Desired output: Create the user's application profile without exposing credentials or allowing the user to assign privileged roles.

Screen 5: Account Status

- Profile completion status.
- Pending or inactive account message, when applicable.
- Continue button for eligible accounts.
- Help or contact option.

Desired output: Prevent unauthorized access when account verification or required profile information is incomplete.

B. Home and Organization Discovery

Screen 6: Home Dashboard

- Personalized greeting.
- Upcoming events.
- Joined organizations.
- Recent announcements.
- Attendance summary.
- Recently earned badges.
- Quick links to organizations, events, and profile.

Desired output: Provide an overview of the user's participation and relevant organizational activities.

Screen 7: Organization Discovery

- Search bar.
- Organization cards.
- Organization logos.
- Organization name and brief description.
- Membership status indicator.
- View organization button.

Desired output: Allow users to discover available organizations.

Screen 8: Organization Details

- Organization logo.
- Organization name.
- Mission, vision, and goals.
- Organization description.
- Membership structure.

- Officers
- Upcoming events.
- Membership status.
- Request to Join button.

Desired output: Present the organization's information and allow eligible users to submit membership requests.

Screen 9: My Organizations

- Joined organizations.
- Membership role.
- Membership status.
- Organization-specific event shortcuts.
- Open organization dashboard.

Desired output: Display all organizations associated with the signed-in user.

Screen 10: Membership Requests

- Pending requests.
- Approved memberships.
- Rejected requests.
- Request date.
- Status and available actions.

Desired output: Let users track their own membership applications and let authorized managers process requests.

C. Events and Registration

Screen 11: Event Discovery

- Upcoming events.
- Ongoing and past events.
- Search and filters.
- Organization selector.
- Event cards containing poster, title, date, time, and venue.

Desired output: Help users find relevant organizational activities.

Screen 12: Event Details

- Event poster or default organization logo.
- Event title.
- Description.
- Date and time.
- Venue.
- Organizer.
- Maximum attendance capacity.
- Registration deadline.
- Registration status.

- Register or cancel registration button, subject to rules.

Desired output: Display complete event information and support eligible registrations.

Screen 13: Event Registration Confirmation

- Event summary.
- Registration requirements.
- Capacity availability.
- Confirm registration button.
- Success or failure message.

Desired output: Create a valid event-registration record without exceeding event capacity or creating duplicate registrations.

Screen 14: My Events

- Registered events.
- Attended events.
- Cancelled registrations.
- Event status.
- Calendar shortcut.

Desired output: Display the user's personal event activity.

Screen 15: Organization Event Management

- Event list.
- Create event button.
- Draft, published, cancelled, and completed statuses.
- Search and filter options.
- Edit, publish, cancel, and archive actions according to permissions.

Desired output: Allow authorized managers and event coordinators to manage events for their assigned organization.

Screen 16: Create or Edit Event

- Event title.
- Description.
- Event poster upload.
- Default organization logo if no poster is provided.
- Date and time.
- Venue.
- Maximum attendees.
- Registration deadline.
- Event visibility.
- Save draft, publish, update, and cancel controls.

Desired output: Validate and save event information. Draft events must not appear as published events to regular users.

D. Membership Management

Screen 17: Organization Member Roster

- Member names.
- Student numbers where authorized.
- Membership roles.
- Membership statuses.
- Search and filters.
- Request review controls.
- Member profile details.

Desired output: Allow managers and authorized secretaries to manage their organization's membership records.

Screen 18: Membership Request Management

- Pending applications.
- Applicant information.
- Application date.
- Approve and reject controls.
- Confirmation dialog.

Desired output: Update membership status while preserving organization-specific permissions.

Screen 19: Member Profile and Engagement

- Approved member details.
- Joined events.
- Attendance summary.
- Earned badges.
- Membership role.
- Available management actions.

Desired output: Help authorized officers understand member participation without exposing unnecessary private data.

Screen 20: Officer and Committee Management

- Current officers.
- Officer position.
- Assigned permissions.
- Committee heads and members.
- Add or update assignments.
- Remove or revoke assignments.

Desired output: Let managers assign responsibilities and restrict access according to each officer's role.

Screen 21: Organization Settings

- Organization name and logo.

- Mission, vision, and goals.
- Membership requirements.
- Organization visibility.
- Membership structure.
- Event approval settings.
- Save changes.

Desired output: Maintain accurate organization information and configuration.

E. Attendance and Gamification

Screen 22: Attendance Verification

- Select organization and event.
- QR scanning area, if QR verification is implemented.
- Manual lookup option for authorized officers.
- Participant verification status.
- Duplicate attendance warning.
- Timestamp and verification result.

Desired output: Record valid attendance only for eligible participants and authorized attendance operators.

Screen 23: Attendance Report

- Event selector.
- Registered participant count.
- Verified attendee count.
- Attendance rate.
- Participant roster.
- Attendance status.
- Date filters and authorized export.

Desired output: Display accurate event attendance statistics.

Screen 24: My Attendance

- Event name.
- Event date.
- Attendance status.
- Verification timestamp.
- Participation summary.

Desired output: Allow each member to view only their own attendance records.

Screen 25: Badge Collection

- Earned badges.
- Badge icons.
- Badge names.
- Badge descriptions.
- Date earned.

- Progress toward available badges, if supported.

Desired output: Encourage continued participation through a digital achievement collection.

Screen 26: Badge Management

- Badge catalog.
- Badge name and description.
- Badge icon.
- Award criteria.
- Active or inactive status.
- Organization-specific badge settings.

Desired output: Allow authorized managers to configure badge definitions and awarding criteria.

Screen 27: Event Ratings

- Event rating summary.
- Star rating.
- Optional written comment.
- Rating submission form.
- Existing rating status.
- Rating moderation tools for authorized personnel.

Desired output: Allow a member to submit only one rating per attended event and prevent unauthorized rating submissions.

F. Calendar and Notifications

Screen 28: Organization Calendar

- Monthly and weekly calendar views.
- Past, present, and future events.
- Event indicators.
- Organization selector.
- Event details when selected.

Desired output: Provide a unified calendar for an organization's activities.

Screen 29: Notifications

- Event announcements.
- Registration confirmations.
- Membership decisions.
- Event updates and cancellations.
- Badge-award notifications.
- Read and unread indicators.

Desired output: Keep users informed about relevant changes.

Screen 30: System Announcements

- Announcement title.
- Content.
- Publication date.
- Audience.
- Priority.
- Read status.

Desired output: Display platform-level notices and organization-specific announcements to their intended audiences.

G. Platform Administration

Screen 31: Platform Admin Dashboard

- Total users.
- Total organizations.
- Active organizations.
- Pending manager-transfer requests.
- Recent platform activity.
- System announcements.

Desired output: Present a platform-level overview without mixing it with organization-specific dashboards.

Screen 32: Organization Administration

- Organization directory.
- Organization status.
- Manager information.
- Organization details.
- Authorized management actions.

Desired output: Let the platform administrator oversee organization records according to platform policies.

Screen 33: Manager Transfer Requests

- Current manager.
- Proposed incoming manager.
- Organization.
- Request date.
- Request status.
- Approve or reject actions for the authorized platform administrator.

Desired output: Transfer management responsibilities securely, preserve an audit trail, and prevent unauthorized self-promotion.

H. Personal Settings and Account Removal

Screen 34: Personal Profile

- Profile photo.
- Full name.

- Student number.
- Year level and section.
- Email.
- Joined organizations.
- Edit permitted fields.

Desired output: Display and maintain the user's own profile.

Screen 35: Account Settings

- Profile preferences.
- Notification preferences.
- Account security information.
- Sign-out button.
- Account removal request.

Desired output: Let users control their own accounts without altering privileged roles.

Screen 36: Account Removal Confirmation

- Explanation of account removal.
- Consequences for membership and historical records.
- Reauthentication or confirmation when required.
- Confirm and cancel actions.

Desired output: Handle account removal securely while following the project's data-retention and privacy policies.

6. Organization Membership Structure

Create a configurable organizational hierarchy that supports common student organization positions:

1. President
2. Vice President for Internal Affairs
3. Vice President for External Affairs
4. Secretary
5. Assistant Secretary
6. Treasurer
7. Auditor
8. Public Information Officer
9. Event Coordinator
10. Committee Heads
11. Committee Members
12. General Members

Treat this as a suggested default structure, not a mandatory list for every organization.

Organizations must be able to customize positions. Store the position title separately from the authorization permissions.

For example, an organization may have an Event Coordinator with event-management permissions, while a Committee Head may only manage assigned committee activities.

The organization manager should be able to assign and revoke permissions within the limits of their authority.

7. Firebase Data Architecture

Design a Firestore database with the following suggested collections.

`users/{uid}`

Fields:

- `uid`
- `email`
- `fullName`
- `studentNumber`
- `yearLevel`
- `section`
- `photoURL`
- `platformRole`
- `accountStatus`
- `createdAt`
- `updatedAt`

The `platformRole` should default to `user`. Elevated platform roles must be provisioned through a trusted administrative process.

`organizations/{organizationId}`

Fields:

- `organizationId`
- `name`
- `description`
- `logoURL`
- `mission`
- `vision`
- `goals`
- `visibility`
- `status`
- `managerUid`
- `membershipRequirements`
- `approvalSettings`
- `createdAt`

- updatedAt

organizations/{organizationId}/memberships/{uid}

Fields:

- uid
- organizationId
- membershipRole
- membershipStatus
- positionTitle
- permissions
- joinedAt
- updatedAt

Keep the organization membership role and status inside the organization-specific membership record. Do not store a single global membership role on the user account.

organizations/{organizationId}/events/{eventId}

Fields:

- eventId
- organizationId
- title
- description
- posterURL
- startAt
- endAt
- venue
- capacity
- registrationDeadline
- registrationCount
- eventStatus
- approvalStatus
- createdBy
- createdAt
- updatedAt

Use timestamps rather than unstructured date and time strings.

organizations/{organizationId}/events/{eventId}/registrations/{uid}

Fields:

- uid

- eventId
- organizationId
- registrationStatus
- registeredAt
- updatedAt

Use a stable participant identifier to prevent duplicate registration records.

organizations/{organizationId}/events/{eventId}/attendance/{uid}

Fields:

- uid
- eventId
- organizationId
- attendanceStatus
- verifiedAt
- verifiedBy
- verificationMethod

Only trusted, authorized operations may create or modify verified attendance.

organizations/{organizationId}/badges/{badgeId}

Fields:

- badgeId
- name
- description
- iconURL
- criteria
- isActive
- createdAt

users/{uid}/badges/{awardId}

Fields:

- badgeId
- organizationId
- eventId
- awardedAt
- awardReason

The awarded badge should be linked to the user and organization. Only a trusted award process may grant badges.

`organizations/{organizationId}/events/{eventId}/ratings/{uid}`

Fields:

- `uid`
- `eventId`
- `organizationId`
- `stars`
- `comment`
- `createdAt`
- `updatedAt`

Use a stable user identifier to enforce one rating per user per event. Rating eligibility must be verified against attendance records.

Additional Collections

Design appropriate structures for:

- `notifications`
- `systemAnnouncements`
- `managerTransferRequests`
- `activityLogs`

Explain where subcollections are preferable and where a top-level collection may simplify cross-organization queries.

Avoid storing duplicated lists of all attended events and all badges on a user document if those lists can grow indefinitely. Query the appropriate records instead.

8. Firebase Business Rules

Implement and document these rules:

1. A user may update only permitted fields in their own profile.
2. A user may not change their own platform role.
3. Organization membership roles and statuses must be managed through authorized operations.
4. Managers may administer only their assigned organizations.
5. Officers may perform only their assigned tasks.
6. Members may access their own private membership, attendance, and badge records.
7. Event registration must respect capacity, deadlines, eligibility, and duplicate prevention.
8. Verified attendance cannot be fabricated by an ordinary client.
9. Badge awards must be generated by trusted logic after the required attendance conditions are satisfied.
10. Ratings require valid attendance and must be limited to one rating per event per user.
11. Account removal must follow the project's retention policy and handle related records appropriately.
12. Manager transfers must be authorized, validated, and logged.
13. Administrative privileges must never be granted through unrestricted public account creation.

14. Firestore Security Rules must enforce permissions even if someone bypasses the application's navigation or edits client-side state.

Use Firebase Cloud Functions or another trusted backend process for privileged operations such as role assignment, event-capacity enforcement, attendance verification, badge awards, and manager transfers when required.

Do not assume that hiding a button is sufficient authorization.

9. Flutter Architecture

Use a layered architecture.

View

- Flutter screens and reusable UI components.
- Displays information and receives user interactions.

ViewModel

- Uses ChangeNotifier.
- Maintains screen state.
- Handles loading, success, error, and empty states.
- Coordinates user actions with repositories.

Repository

- Provides application-level data operations.
- Separates business logic from UI implementation.
- Coordinates data sources.

Service

- Communicates with Firebase Authentication, Cloud Firestore, Cloud Storage, and trusted backend endpoints.

Model

- Represents User, Organization, Membership, Event, Registration, Attendance, Badge, Rating, and Notification objects.

Use Provider for dependency injection and state updates, and go_router for navigation and route protection.

Organize the project into feature folders, such as:

- `auth`
- `home`
- `organizations`
- `memberships`
- `events`

- `attendance`
- `badges`
- `ratings`
- `calendar`
- `notifications`
- `admin`
- `profile`
- `settings`

Provide a recommended project directory tree and explain the responsibility of each layer.

10. Application Navigation

Design a bottom navigation bar for regular users with:

- Home
- Organizations
- Events
- Badges
- Profile

Provide role-dependent management entry points for managers, officers, and platform administrators.

Use an organization selector whenever an authorized user manages more than one organization.

Create a sitemap showing all routes and which roles can access them.

Unauthenticated users should be redirected to sign-in when opening protected screens. Unauthorized users must receive an appropriate access-denied message.

11. System Flowcharts

Use the uploaded BarkadaBase PDF as the visual reference for flowchart presentation. Adapt the flowcharts to the OrgLink mobile application and Firebase architecture.

Generate the following Mermaid diagrams:

1. Overall application flow.
2. Google sign-in and profile completion.
3. Role-based dashboard routing.
4. Organization creation and manager assignment.
5. Membership request and approval.
6. Event creation and publication.
7. Event registration and capacity validation.
8. Attendance verification.
9. Automatic badge awarding.
10. Event rating submission.
11. Manager transfer.

12. Account removal.

Core Authentication Flow

1. User opens OrgLink.
2. Firebase Authentication checks the sign-in state.
3. User signs in using Google.
4. The application obtains the authenticated UID.
5. The application loads the profile and authoritative platform-role information.
6. The application checks the account status.
7. The user is directed to the appropriate screen.
8. Organization-specific permissions are checked when accessing an organization's functions.
9. Firestore Security Rules and trusted backend checks enforce the operation.

Automatic Badge Flow

1. An event is published.
2. A member registers for the event.
3. An authorized officer verifies the member's attendance.
4. A trusted process validates the event, member, and attendance record.
5. The process checks the organization's badge criteria.
6. The process creates the badge award if the criteria are satisfied.
7. Duplicate awards are prevented.
8. The member's badge collection is updated.
9. A notification may be generated.

12. Asset Inventory

Produce a complete table with the following columns:

- Asset name.
- Purpose.
- File type.
- Suggested filename.
- Dimensions or specifications.
- Screens where used.
- Required or optional.
- Source or creation method.

Include:

Branding Assets

- OrgLink logo.
- App icon.
- Splash-screen logo.
- Default student avatar.
- Organization logo placeholder.
- Event poster placeholder.

- Empty-state illustrations.

Interface Assets

- Home icon.
- Organization icon.
- Event icon.
- Attendance icon.
- Badge icon.
- Calendar icon.
- Notification icon.
- Settings icon.
- Search, filter, edit, delete, and confirmation icons.
- Loading indicators.
- Success and error icons.

Gamification Assets

- Badge catalog icons.
- First-event badge.
- Active-participant badge.
- Attendance milestone badge.
- Participation streak badge, if implemented.
- Badge-award animation, if feasible.

Development Assets

- Flutter source files.
- Dart model files.
- ViewModel files.
- Repository interfaces and implementations.
- Firebase service files.
- Firebase configuration files.
- Firestore Security Rules.
- Storage Security Rules, if Storage is used.
- Firebase Cloud Functions, if needed.
- Firestore index configuration.
- Firebase Emulator configuration.
- Git ignore configuration.
- Environment setup documentation.
- README.
- Testing fixtures and sample data.

Do not include production credentials, service-account private keys, or passwords in the asset list or application source.

13. Page-by-Page Functional Specification

For every screen, create a detailed table with these columns:

1. Screen name.
2. User role.
3. Purpose.
4. UI components.
5. Available actions.
6. ViewModel responsibilities.
7. Repository operations.
8. Firebase services used.
9. Firestore collection paths.
10. Permission requirements.
11. Validation rules.
12. Desired output.
13. Loading state.
14. Empty state.
15. Error state.
16. Success state.

Every button, form, card action, and navigation item must have a defined function.

Include both normal and exceptional outcomes, such as duplicate registration, full event capacity, rejected membership, failed sign-in, unauthorized role changes, and duplicate attendance scans.

14. Design System

Create an initial mobile UI design system suitable for a university organization application.

Suggested style:

- Primary: deep blue.
- Secondary: medium blue.
- Accent: teal or restrained gold.
- Background: white and light gray.
- Typography: clean, readable sans-serif.
- Cards: rounded corners and consistent spacing.
- Buttons: clearly distinguish primary, secondary, and destructive actions.
- Badges: distinct icons with accessible labels.
- Navigation: simple bottom navigation for members and role-specific management sections.
- Forms: labeled inputs and visible validation messages.

Provide low-fidelity wireframes first. Do not prioritize decorative visuals over usability.

Use consistent Android screen dimensions and provide representative designs for small and large phone layouts.

15. Required Deliverables

Present the final result in this order:

Deliverable 1: Project overview and assumptions.

Deliverable 2: Role hierarchy and permission matrix.

Deliverable 3: Complete application sitemap and route map.

Deliverable 4: Low-fidelity wireframes for all 36 specified screens.

Deliverable 5: Shared components and design system.

Deliverable 6: Complete asset inventory with filenames and specifications.

Deliverable 7: Page-by-page functionality and desired outputs.

Deliverable 8: Firestore database structure and example document schemas.

Deliverable 9: Firebase Authentication, authorization, and security architecture.

Deliverable 10: Mermaid flowcharts for all major application workflows.

Deliverable 11: Flutter project directory structure using Provider, ChangeNotifier, repositories, services, and models.

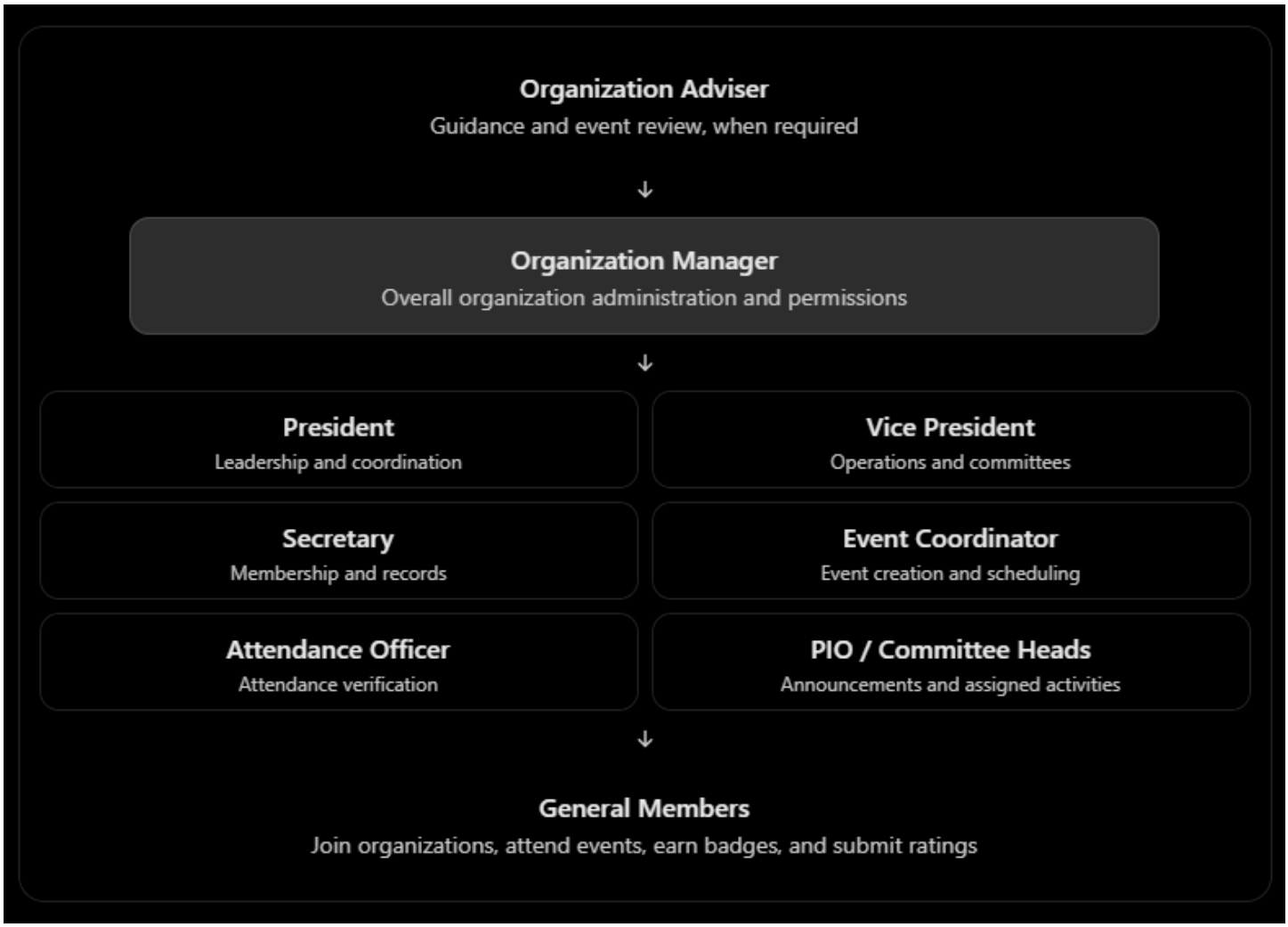
Deliverable 12: Testing checklist for permissions, registration, attendance, badge awards, ratings, manager transfers, and account removal.

Deliverable 13: Development roadmap separating the minimum viable prototype from optional future enhancements.

16. Final Instructions

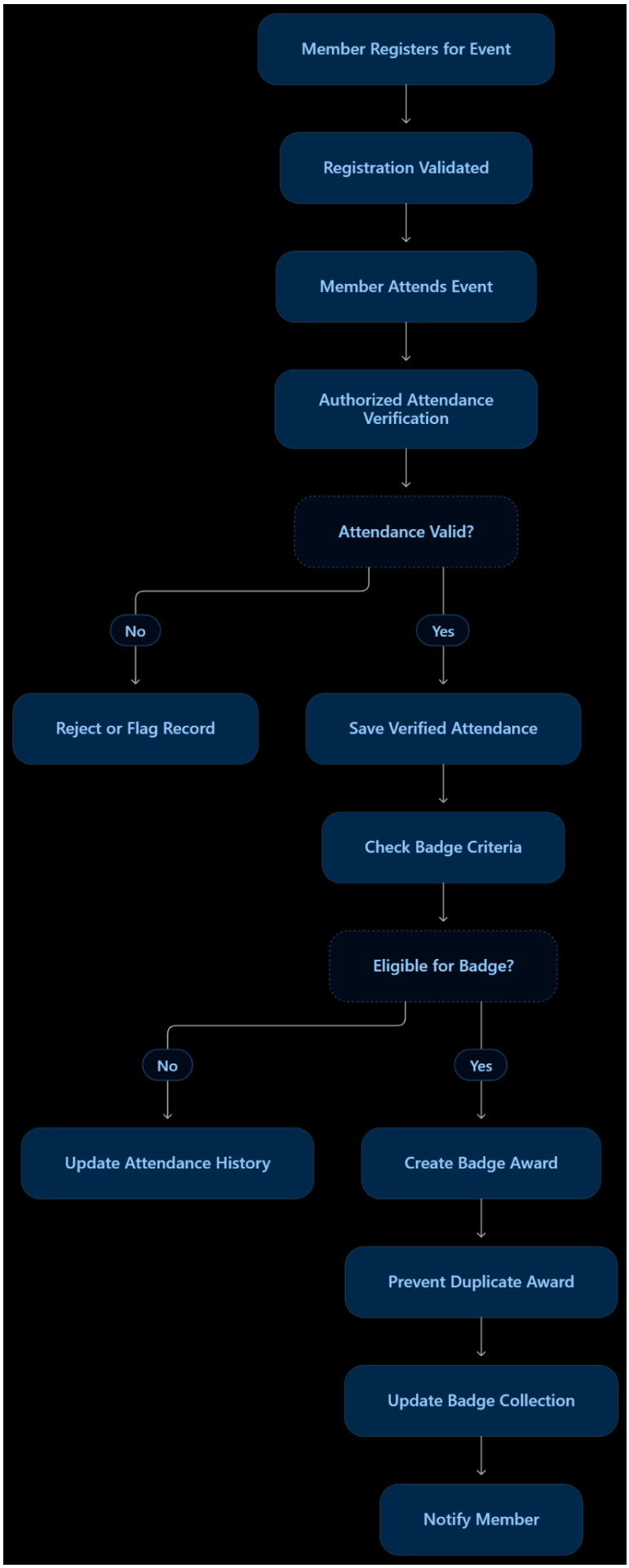
- Use the uploaded BarkadaBase PDF as a reference for visualization and flowchart format, but adapt the system to OrgLink's updated concept.
- Design an Android mobile application rather than a desktop-first website.
- Treat the specified Firebase architecture as the intended implementation.
- Do not claim that proposed features are already implemented.
- Label assumptions and optional enhancements.
- Prioritize a feasible student capstone implementation.
- Make Student/Member, Officer, Manager, and Platform Administrator permissions explicit.
- Keep platform roles separate from organization membership roles.
- Ensure that every page has a clear function and expected output.
- Include actual wireframes, diagrams, tables, and schemas rather than descriptions alone.
- Ensure that the design supports multiple organizations with different memberships and permissions.
- Use secure backend validation for privileged operations.
- Keep the UI, ViewModels, repositories, services, and models logically separated.

The final deliverable must be a complete initial design blueprint that can be used to develop OrgLink in Flutter and Dart with Firebase, test it using the Firebase Emulator Suite, and present it as a Computer Engineering capstone project.



Collection	Important Attributes
Users	UID, email, full name, student number, year level, section, photo URL, platform role, account status
Organizations	Organization ID, name, logo, description, mission, vision, goals, manager UID, visibility, settings
Memberships	User UID, organization ID, membership role, membership status, position title, permissions, joined date
Events	Event ID, organization ID, title, description, poster URL, start/end time, venue, capacity, registration deadline, event status
Registrations	User UID, event ID, registration status, registration timestamp
Attendance	User UID, event ID, attendance status, verification timestamp, verified by, verification method

Badges	Badge ID, organization ID, name, description, icon, awarding criteria
Badge Awards	User UID, badge ID, organization ID, event ID, award date
Ratings	User UID, event ID, organization ID, star rating, comment, submission timestamp
Notifications	Recipient UID, type, message or related record, read status, timestamp


Member Registers for Event
Registration Validated
Member Attends Event
Authorized Attendance Verification
Attendance Valid?
No
Yes
Reject or Flag Record
Save Verified Attendance
Check Badge Criteria
Eligible for Badge?
No
Yes
Update Attendance History
Create Badge Award
Prevent Duplicate Award
Update Badge Collection
Notify Member

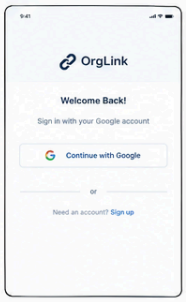
OrgLink

Student Organization Management App
Initial Wireframes

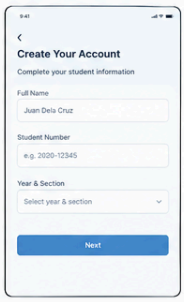
1. Splash Screen



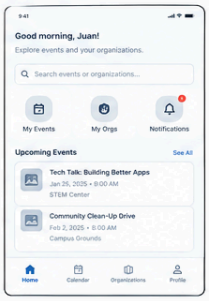
2. Login / Sign Up



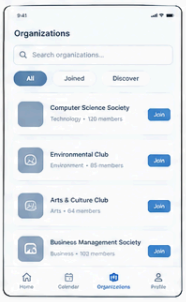
3. Student Identity (Sign Up)



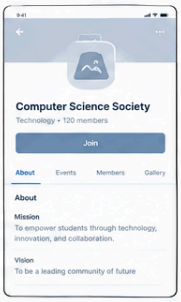
4. Home / Dashboard



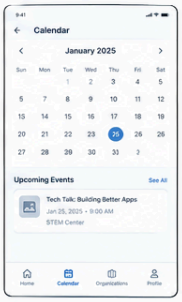
5. Organization List / Browse



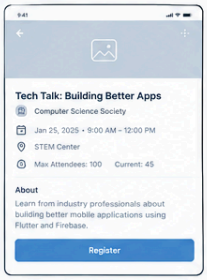
6. Organization Profile



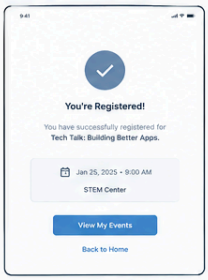
7. Organization Calendar



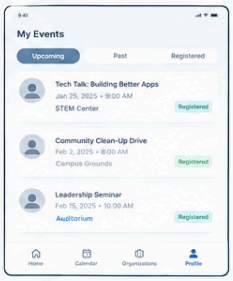
8. Event Details



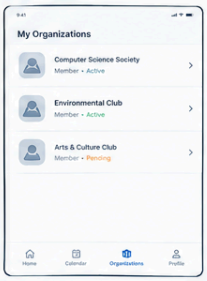
9. Event Registration (Confirmation)



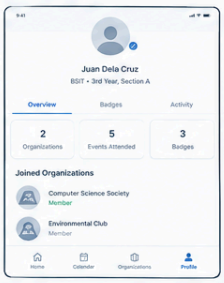
10. My Events



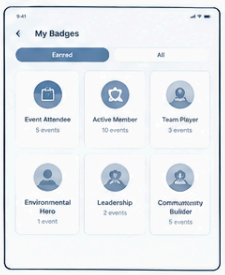
11. My Organizations



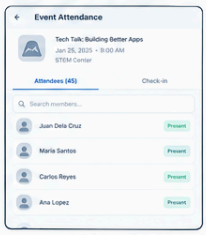
12. Member Profile



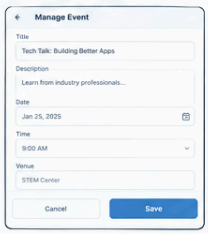
13. Badges



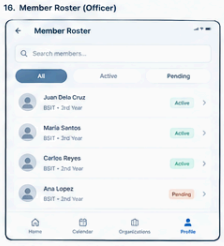
14. Attendance Check (Officer View)



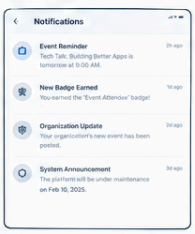
15. Event Management (Officer)



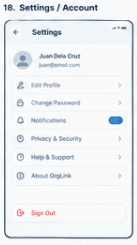
16. Member Roster (Officer)



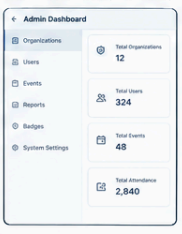
17. Notifications



18. Settings / Account



19. Role-based Admin Panel (Platform Admin)



20. Organization Management (Manager)

